Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЕТ
к лабораторной работе №3
на тему:
«ИДЕНТИФИКАЦИЯ И АУТЕНТИФИКАЦИЯ ПОЛЬЗОВАТЕЛЕЙ. ПРОТОКОЛ KERBEROS»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артём Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Краткие теоритичесие сведения 4
3 Выполнение работы 6
Вывод 8
Приложение А (справочное) Исходный код 9

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Идентификация и аутентификация пользователей через протокол Kerberos. Необходимо программно реализовать протокол Kerberos.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Kerberos – это протокол, который проверяет удостоверения пользователей или узлов с помощью системы цифровых «билетов». Он использует криптографию с секретным ключом и доверенную третью сторону для проверки личности пользователей и аутентификации клиент-серверных приложений.

Протокол Kerberos был изначально разработан в Массачусетском технологическом институте (MIT) в 1988 году, чтобы университет мог безопасно аутентифицировать сетевых пользователей и разрешать им доступ к определенным ресурсам, таким как хранилище и базы данных.

Имя Kerberos происходит из греческой мифологии. В ней Kerberos (Цербер) – трехголовый пес, охраняющий врата в мир мертвых. Трём головам Цербера в протоколе соответствуют три участника безопасной связи: клиент, сервер и центр распределения ключей Kerberos (KDC), который выдает «билеты» Kerberos.

На сегодняшний день Kerberos является одним из наиболее широко используемых сетевых протоколов аутентификации. Он часто используется для поддержки единого входа в крупных корпоративных сетях, является методом аутентификации по умолчанию в Windows и играет неотъемлемую роль в Windows Active Directory.

Рассмотрим структуру протокола Kerberos на рисунке 2.1.

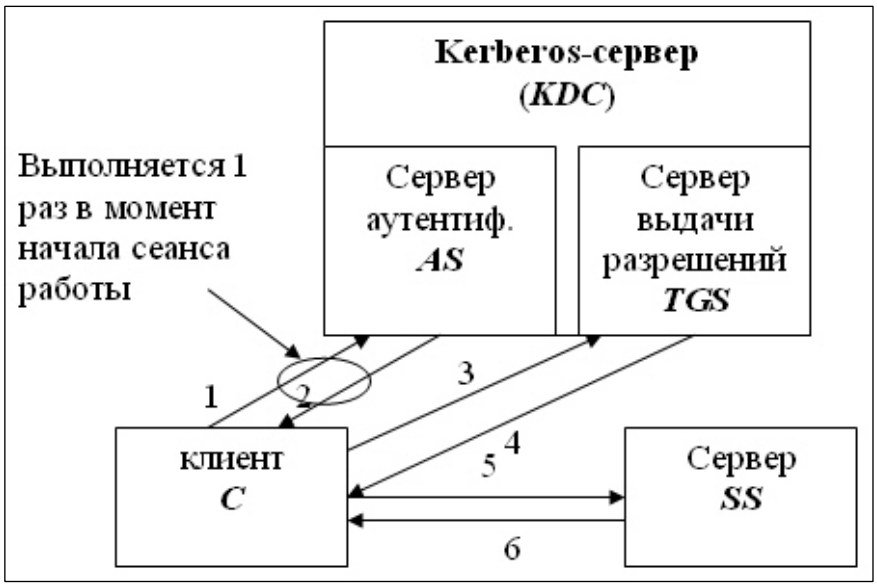

Рисунок 2.1 – Структура протокола Kerberos

Описание работы протокола Kerberos:

1 Клиент C направляет запрос серверу аутентификации AS с целью получения билета на выдачу билетов TGT. В запросе указывается идентификатор клиента и запрашиваемый сервис TGS.

2 Сервер AS проверяет наличие клиента в своей базе данных. Если клиент найден, AS генерирует сессионный ключ для последующего взаимодействия клиента с TGS и создаёт TGT, содержащий идентификатор клиента и этот сессионный ключ. TGT зашифровывается секретным ключом TGS. Затем AS отправляет клиенту ответ, зашифрованный паролем клиента, включающий TGT и сессионный ключ. Если клиент не найден, процесс останавливается.

3 Клиент расшифровывает ответ своим паролем, извлекая TGT и сессионный ключ для TGS. Далее клиент формирует запрос к серверу выдачи разрешений TGS на получение билета для доступа к конкретному сервису SS. Запрос включает TGT и аутентификатор, зашифрованный сессионным ключом TGS (содержащий идентификатор клиента и временную метку).

4 Сервер TGS расшифровывает TGT своим ключом, получает сессионный ключ и проверяет аутентификатор. При успешной проверке TGS генерирует новый сессионный ключ для связи клиента с сервером SS и создаёт билет для SS, содержащий идентификатор клиента и этот новый ключ. Билет зашифровывается секретным ключом SS. Ответ клиенту (содержащий билет для SS и новый сессионный ключ) зашифровывается сессионным ключом TGS.

5 Клиент расшифровывает ответ, получает билет для SS и соответствующий сессионный ключ. Для обращения к серверу SS клиент формирует аутентификатор, зашифрованный сессионным ключом SS, и отправляет его вместе с билетом серверу SS.

6 Сервер SS расшифровывает билет своим секретным ключом, извлекает сессионный ключ и проверяет аутентификатор. Если проверка проходит успешно (временная метка актуальна, идентификатор совпадает), сервер предоставляет клиенту доступ к ресурсу. В некоторых реализациях сервер также отправляет клиенту подтверждение, зашифрованное сессионным ключом, для взаимной аутентификации.

3 ВЫПОЛНЕНИЕ РАБОТЫ

В ходе выполнения лабораторной работы была реализована программная модель протокола аутентификации Kerberos. В модели взаимодействуют три основных участника безопасной связи: клиент, сервер ресурсов и центр распределения ключей Kerberos (KDC). KDC, выступающий в роли доверенной третьей стороны, реализован как совокупность двух сервисов – сервера аутентификации (AS) и сервера выдачи билетов (TGS). Клиентская часть программы запрашивает и обрабатывает билеты, сервер ресурсов проверяет подлинность предъявленных учётных данных. Вся программная реализация выполнена на языке Python с использованием библиотеки cryptography.fernet для симметричного шифрования. Для хранения учётных данных пользователей и секретных ключей сервисов созданы модули базы данных (db.py), а для операций шифрования – утилита (crypto_utils.py).

Взаимодействие между участниками было разделено на три логических этапа, которые соответствуют классической схеме Kerberos. На первом этапе клиент отправил запрос в сервис аутентификации AS (часть KDC). AS проверил наличие пользователя в базе, сгенерировал сессионный ключ для общения клиента с TGS и создал билет TGT, зашифровав его секретным ключом TGS. Весь ответ клиенту был зашифрован паролем пользователя. После расшифровки ответа своим паролем клиент получил сессионный ключ TGS и сам билет TGT.

На втором этапе клиент использовал полученный TGT для запроса билета на доступ к сервису file_storage у сервера TGS (второй компонент KDC). TGS расшифровал TGT своим ключом, извлёк из него идентификатор клиента и сессионный ключ, после чего сгенерировал новый сессионный ключ для связи клиента с сервером ресурсов. Этот новый ключ был помещён в билет для сервиса, зашифрованный ключом самого сервера file_storage.

Заключительный, третий этап, заключался в обращении клиента непосредственно к серверу ресурсов. Клиент создал аутентификатор, содержащий своё имя и текущую временную метку, зашифровав его сессионным ключом сервиса. Затем клиент отправил на сервер этот аутентификатор вместе с билетом, полученным от TGS. Сервер file_storage расшифровал билет своим секретным ключом, извлёк из него сессионный ключ, с помощью которого успешно расшифровал аутентификатор. Проверка временной метки подтвердила, что запрос не является повторной атакой. В результате сервер вывел сообщение об успехе, что показано на рисунке 3.1.

```
--- 1. Запрос к AS(Authentication Service) - Логин: artem_kokhan ---
TGT получен и расшифрован.

--- 2. Запрос к TGS(Ticket Granting Service) - Сервис: file_storage ---
Билет к сервису получен.

--- 3. Обращение к Серверу ---
[Server] Успех! Доступ разрешен для artem_kokhan
Финальный результат: АУТЕНТИФИКАЦИЯ ПРОЙДЕНА
```

Рисунок 3.1 – Результат работы программы

ВЫВОД

В результате выполнения лабораторной работы была разработана программная реализация протокола Kerberos, наглядно демонстрирующая взаимодействие трёх ключевых участников: клиента, сервера ресурсов и центра распределения ключей KDC. На практике были отработаны механизмы выдачи билетов, обмена сессионными ключами и проверки подлинности клиента на стороне сервера с использованием временных меток. Корректность работы протокола подтверждена аутентификацией пользователя и предоставлением доступа к целевому сервису.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация crypto_utils.py

```
import base64
import hashlib
from cryptography.fernet import Fernet

def get_fernet_key(secret: str):
    """Создает ключ Fernet на основе строки (пароля)"""
    key = hashlib.sha256(secret.encode()).digest()
    return Fernet(base64.urlsafe_b64encode(key))

def encrypt(key_source, data: str):
    """Шифрует строку. key_source может быть строкой или уже объектом Fernet"""
    f = get_fernet_key(key_source) if isinstance(key_source, str) else
Fernet(key_source)
    return f.encrypt(data.encode())

def decrypt(key_source, token):
    """Расшифровывает данные в строку"""
    f = get_fernet_key(key_source) if isinstance(key_source, str) else
Fernet(key_source)
    return f.decrypt(token).decode()
```

Листинг А.2 – Реализация as_service.py

```
from kerberos import crypto_utils
from kerberos.db import USER_DB, TGS_SECRET_KEY
from cryptography.fernet import Fernet


def get_tgt(client_id):
    if client_id not in USER_DB:
        raise Exception("Пользователь не найден")

    session_key_tgs = Fernet.generate_key()

    tgt_data = f"{client_id}|{session_key_tgs.decode()}"
    tgt = crypto_utils.encrypt(TGS_SECRET_KEY, tgt_data)

    client_password = USER_DB[client_id]
    response_data = f"{session_key_tgs.decode()}|{tgt.decode()}"

    return crypto_utils.encrypt(client_password, response_data)
```

Листинг А.3 – Реализация tgs_service.py

```
from kerberos import crypto_utils
from kerberos.db import SERVICE_DB, TGS_SECRET_KEY
from cryptography.fernet import Fernet


def get_service_ticket(tgt_encrypted, service_id):
    if service_id not in SERVICE_DB:
        raise Exception("Сервис не существует")
```

```
    tgt_data = crypto_utils.decrypt(TGS_SECRET_KEY, tgt_encrypted)
    client_id, session_key_tgs = tgt_data.split("|")

    service_session_key = Fernet.generate_key()

    ticket_data = f"{client_id}|{service_session_key.decode()}"
    service_ticket = crypto_utils.encrypt(SERVICE_DB[service_id], ticket_data)

    response_data                                                          =
f"{service_session_key.decode()}|{service_ticket.decode()}"
    return crypto_utils.encrypt(session_key_tgs, response_data)
```

Листинг А.4 – Реализация server.py

```
from kerberos import crypto_utils
from kerberos.db import SERVICE_DB
from datetime import datetime

SERVICE_ID = "file_storage"
SERVER_SECRET = SERVICE_DB[SERVICE_ID]


def validate_request(ticket_encrypted, auth_encrypted):
    try:
        ticket_data = crypto_utils.decrypt(SERVER_SECRET, ticket_encrypted)
        client_id, session_key_service = ticket_data.split("|")

        auth_data = crypto_utils.decrypt(session_key_service, auth_encrypted)
        auth_client_id, timestamp_str = auth_data.split("|")

        request_time = datetime.strptime(timestamp_str, "%Y-%m-%d %H:%M:%S")
        time_diff = (datetime.now() - request_time).total_seconds()

        if abs(time_diff) > 300:
            print("[Server] Ошибка: Слишком старый запрос (Replay Attack?)")
            return False

        print(f"[Server] Успех! Доступ разрешен для {auth_client_id}")
        return True
    except Exception as e:
        print(f"[Server] Ошибка проверки: {e}")
        return False
```

Листинг А.5 – Реализация client.py

```
from server import server
from datetime import datetime
from kerberos import crypto_utils, tgs_service, as_service


def run_auth_flow():
    # Данные пользователя
    user = "artem_kokhan"
    password = "artem_kokhan_password_123"
    target_service = "file_storage"

    print(f"--- 1. Запрос к AS(Authentication Service) - Логин: {user} ---")
    try:
        as_resp = as_service.get_tgt(user)
        # Клиент расшифровывает ответ своим паролем
        s_key_tgs, tgt = crypto_utils.decrypt(password, as_resp).split("|")
        print("TGT получен и расшифрован.")
```

```
        print(f"\n--- 2. Запрос к TGS(Ticket Granting Service) - Сервис: {target_service} ---")
        tgs_resp = tgs_service.get_service_ticket(tgt, target_service)
        # Расшифровываем ключом сессии TGS
        s_key_service, service_ticket = crypto_utils.decrypt(s_key_tgs, tgs_resp).split("|")
        print("Билет к сервису получен.")

        print("\n--- 3. Обращение к Серверу ---")
        # Создаем аутентификатор (время + ID)
        curr_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        authenticator = crypto_utils.encrypt(s_key_service, f"{user}|{curr_time}")

        if server.validate_request(service_ticket, authenticator):
            print("Результат: АУТЕНТИФИКАЦИЯ ПРОЙДЕНА")
        else:
            print("Результат: ОТКАЗАНО")

    except Exception as e:
        print(f"Произошла ошибка в kerberos: {e}")


if __name__ == "__main__":
    run_auth_flow()
```